Baze de Date

Indrumar de laborator

Laboratorul 1

Introducere. Proiectarea bazelor de date

Obiective

1. Sa descriem ce este o baza de date relationala si care sunt componentele acesteia.
2. Sa identificam tabelele ce formeaza o baza de date, in exercitii practice.
3. Sa identificam campurile unui tabel, conform exemplelor analizate.
4. Sa stabilim cheia primara a unui tabel, conform contextului dat.
5. Sa identificam relatiile intre tabele si sa justificam alegerile.

Notiuni teoretice

I Introducere

I.1 Definitie

Din punct de vedere conceptual, o baza de date poate fi privita ca un depozit de organizat date.

Din punct de vedere tehnic, o baza de date este o colectie organizata de obiecte (tabele, vederi, proceduri stocate, functii).

I.2 Aplicatii ale bazelor de date

Multe aplicatii, indiferent de limbajul in care au fost implementate, au nevoie sa sa stocheze, sa organizeze si sa acceseze o baza de date.

De exemplu, intr-o aplicatie e-commerce, produsele, cosurile de cumparaturi, comenzile, datele clientilor, sunt stocate in baza de date. Acest lucru ajuta la gestionarea milioanelor de produse (e.g. Amazon), astfel incat informatiile sa nu fie pierdute, inconsistente, duplicate, si asa mai departe.

Pentru banci si servicii financiare, bazele de date sunt folosite pentru gestionarea conturilor clientilor, a tranzactiilor, a soldurilor. In acest fel se permite accesul securizat, in timp real, la informatii, ajutand la detectarea fraudelor prin interogari complexe.

In cazul retelelor sociale, o baza de date poate permite gestionarea profilului utilizatorilor, prietenilor, postarilor, sau a comentariilor. Interogarile realizate permit accesul rapid la un feed personalizat, iar baza de date ajuta la stocarea datelor multimedia (poze, video, mesaje).

I.3 Diferenta dintre date si informatie

Atunci cand se lucreaza cu baze de date, este important sa fie diferentiate cele doua notiuni: date si informatie.

Datele reprezinta valorile brute, neprelucrate, prezente in bazele de date. Acestea pot fi privite ca fiind materia prima, cum ar fi “120”, “Aleea Republicii”, “2025-06-21”.

Informatiile sunt datele prelucrate, carora li s-a oferit un context, pentru a transmite o idee. Acestea pot fi interpretate ca un produs finit, utilizat in luarea deciziilor.

Folosind datele de mai sus, informatia transmisa poate fi: Pe Aleea Republicii, numarul 120, in data de 21 Iunie 2025, s-a livrat un colet.

Astfel, se poate deduce rolul bazelor de date, sa transforme datele brute in informatie care are sens, este usor accesibila si corecta.

I.4 Sistemul de Gestiune a Bazelor de Date

Pentru fi asigurata interactiunea cu baza de date, utilizatorul nu lucreaza direct cu fisiere brute, ci prin intermediul unui program, denumit Sistemul de Gestiune a Bazelor de Date (SGBD), in engleza: Database Management System (DBMS). Acest software dedicat permite proiectarea si implementarea bazelor de date, stocarea si organizarea datelor, actualizarea si extragerea datelor prin interogari, precum si accesul controlat la date, pentru asigurarea securitatii.

Cateva exemple de SGBD sunt:

- Open-source: MySQL, PostgreSQL, SQLite, MariaDB (nu necesita licenta pentru a fi utilizata)
- Comerciale: Microsoft SQL Server, Oracle Database, IBM Db2 (acces prin achizitionarea unei licente)

II Concepte fundamentale

Inainte de a invata cum se stocheaza datele intr-o baza de date relationala, este necesara cunoasterea a trei notiuni de baza: entitati, atribute, valori.

Entitate = obiect din realitatea inconjuratoare, de exemplu, student, profesor sau un curs, carte, care formeaza o clasa de obiecte. Intr-o baza dee date relationala, entitatea este reprezentata de un tabel.

- Exemplu: Enitatea Clienti descrie la nivel conceptual toti clientii posibili, fiecare instanta a clasei reprezentand un client individual.

Atribute = caracteristici ale unei entitati. Intr-o baza de date relationala, atributul corespunde unei coloane (camp) din tabel.

- Exemplu: Instantele entitatii Client vor avea atribute identice care sa le reprezinte: Nume, Prenume, Adresa, E-mail, Telefon.

Valoare = instanta concreta a unui atribut, reprezentant informatia stocata. Intr-o baza de date relationala, valorile sunt stocate in celulele tabelului.

- Exemplu: O instanta a entitatii clienti poate contine urmatoarele valori pentru atributele care o caracterizeaza: Popescu, Radu, Strada SQL, radup@yahoo.com, 0744000111, iar o alta instanta poate avea valori complet diferite: Ionescu, Anca, Strada Server, anca@gmail.com, 0722666333.

II.1 Anatomia unei baze de date

O baza de date este formata din unul sau mai multe tabele.

Tabelul

Tabelul = structuri bidimensionale, formate din randuri si coloane, care stocheaza datele. Fiecare tabel este folosit pentru a reprezenta un singur subiect/o singura entitate (Client, Produse, Student, Cursuri) si contine informatii caracteristice acestuia.

Un tabel este format din campuri si inregistrari.

- Campurile = indica datele care pot fi stocate in tabelul respectiv.
- Inregistrarile = valori efective ale campurilor.

Fiecare tabel contine cel putin un **camp** care identifica in mod **unic** fiecare inregistrare, camp cunoscut sub denumirea de **cheie primara**.

Elementele tabelului ideal

- Tabelul reprezinta un singur subiect, care poate fi un obiect sau un eveniment, cu un nume clar si sugestiv.
- Tabelul are o singura cheie primara, cu caracter unic, nenul, stabil in timp.
- Tabelul nu contine campuri cu mai multe parti sau cu mai multe valori (de exemplu, “Adresa” formata din Strada, Numar, Oras).
- Tabelul nu contine campuri calculate (acestea rezulta din interogari).
- Tabelul nu contine campuri duplicate inutile
- Tabelul contine doar o cantitate minima de date redundante

Exemplu de tabel Clienti

ID	**Prenume client**	**Nume client**	**CNP**	**Telefon**	……
100	Ana	Ionescu	2820216627918	0711123456	
101	Mihai	Popescu	1920415647718	0789765345	
102	Bogdan	Florescu	1750212697717	0723674660	
…					

Campurile

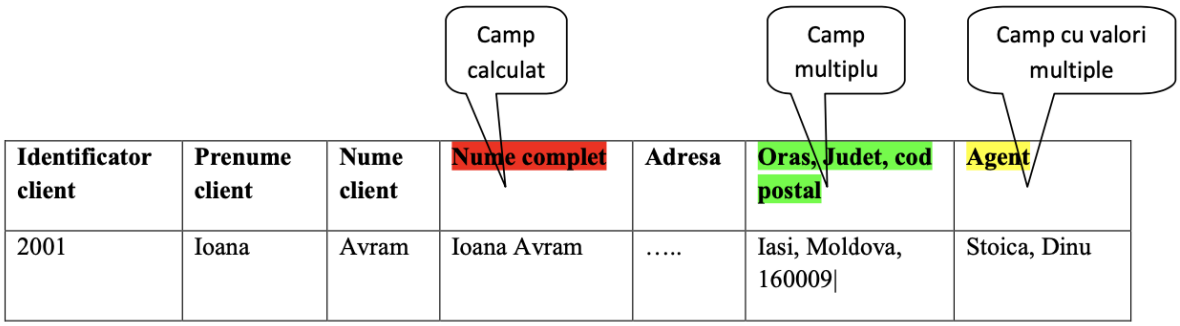
Campuri = denumite si atribute, reprezinta cea mai mica structura din baza de date, aratand o caracteristica a subiectului tabelului. Fiecare camp descrie o trasatura particulara a entitatii pe care o caracterizeaza (ex., numele unei persoane). Campurile reprezinta coloanele unui tabel.

De exemplu, pe baza tabelului de mai sus, entitatea Clienti are atributele: prenume, nume, CNP, telefon. Mai pot fi completate de adresa, e-mail, etc. Fiecare dintre aceste campuri este caracterizat de un tip de date. Acest tip de date va determina ce informatii pot fi inregistrate in campul respectiv. Fiecare camp dintr-o baza de date corect proiectata contine o singura valoare, iar numele sau va identifica tipul de valoare pe care il contine.

Intr-o baza de date slab proiectata sau proiectata inadecvat, se vor intalni alte 3 tipuri de campuri:

- camp multiplu (multipart field) – contine doua sau mai multe elemente distincte in cadrul valorii acestuia.
- camp cu valori multiple (multivalued field) – contine mai multe aparitii ale aceluiasi tip de valoare.
- camp calculat – contine o valoare de text concatenat sau rezultatul unei expresii matematice.

Figura urmatoare prezinta un tabel cu exemple din fiecare dintre cele trei tipuri de campuri:



Identificator client	Prenume client	Nume client	Nume complet	Adresa	Oras, Judet, cod postal	Agent
2001	Ioana	Avram	Ioana Avram	…..	Iasi, Moldova, 160009	Stoica, Dinu

Elementele campului ideal

- Fiecare camp reprezinta o caracteristica distincta a subiectului tabelului, avand un nume clar si sugestiv (Nume, Data_Nasterii, Pret).
- Campul contine o singura valoare/informatie, evitand concatenarile de tipul Nume Complet = “Nume+Prenume”.
- Campul nu poate fi descompus in componente mai mici, nu poate contine o valoare calculata sau obtinuta prin concatenarea altor campuri (Nume, Initiala_tatalui, Prenume vor fi campuri separate).
- Campul este unic in cadrul intregii structuri a bazei de date. De exemplu, daca doua tabele diferite au campul “Cod”, insa unul reprezinta cod produs, iar altul este cod client, avest lucru va genera confuzie; ar trebui denumite CodProdus si CodClient.
- Campul isi pastreaza caracteristicile cand apare in mai multe tabele. De exemplu, daca in tabelul Clienti exista campul ID_Client = INT(4), iar in tabelul Comenzi exista ID_Client = VARCHAR(10), atunci pot genera erori.

Inregistrarile

Inregistrarile = instante individuale ale unei entitati, fiind alcatuita din intregul set de campuri ale unui tabel, indiferent daca acele campuri contin sau nu valori. Inregistrarile reprezinta randurile unui tabel.

Fiecare inregistrare este identificata in mod unic prin valoarea unica a cheii primare (exemplu: ID).

ID	**Prenume client**	**Nume client**	**CNP**	**Telefon**	……
100	Ana	Ionescu	2820216627918	0711123456	
101	Mihai	Popescu	1920415647718	0789765345	
102	Bogdan	Florescu	1750212697717	0723674660	
…					

In tabelul de mai sus, fiecare inregistrare reprezinta un client unic din cadrul tabelului. Fiecare client este identificat in mod unic prin campul ID. La randul ei, fiecare inregistrare include toate campurile din tabel, iar fiecare camp descrie un aspect al clientului reprezentat de inregistrare.

Elementele inregistarii ideale

- Are o cheie primara (ID) unica, care o diferentiaza de celelalte randuri.
- Are toate campurile obligatorii completate, iar lipsa valorilor este permisa conform constrangerilor.
- Respecta formatul si tipul definit (de exemplu, DataAngajare are un format valid: mm/dd/yyyy; Salariu este de tip numeric, pozitiv).
- Este consistenta, evitand contradictiile intre date (de exemplu, clientul cu vechimea de 10 ani are varsta de 5 ani).

II.2 Cheile unui tabel si rolul lor in bazele de date

Cheile sunt acele campuri speciale care indeplinesc roluri foarte bine definite in cadrul unui tabel, iar tipul cheii defineste rolul acesteia in interiorul tabelului. Acestea ajuta la identificarea fiecarui rand (inregistrare) din tabel si la stabilirea relatiilor intre tabele. Cele mai importante tipuri de chei sunt cheia primara si cheia externa.

Tipuri de chei

Cheia primara (primary key - PK)

O cheie primara este un camp sau un grup de campuri care identifica in mod unic fiecare inregistrare din cadrul unui tabel.

Fiecare tabel dintr-o baza de date trebuie sa contina o cheie primara!

Pentru a asigura performantele aplicatiei, cheia primara trebuie sa fie:

- minimala - formata dintr-un numar minim de campuri; nu contine coloane in plus daca nu ajuta la identificare.
- stabila - nu trebuie sa se schimbe in timp sau se schimba rar.
- unica - fiecare valoare trebuie sa fie unica; sa nu existe doua randuri cu aceeasi cheie primara.
- non-null - nu poate contine valori NULL.

Considerand exemplul unui tabel ce modeleaza angajatii unei companii:

Prenume client	**Nume client**	**CNP**	**Telefon**	**Data Angajarii**
Ana	Ionescu	2820216627918	0711123456	02/15/2024
Mihai	Popescu	1920415647718	0789765345	08/01/2025
Bogdan	Florescu	1750212697717	0723674660	11/03/2026

Un astfel de tabel este format din campuri ce definesc un angajat cum ar fi: nume, prenume, CNP, data angajarii, salariu etc. Pentru a stabili cheia primara a unui astfel de tabel trebuie sa fie selectat campul sau asocieri de campuri ce identifica in mod unic fiecare dintre angajatii companiei.

Alegerea cheii primare

Variante:

1. **Asocierea nume + prenume** - aceasta reprezinta o versiune valida pana in momentul in care compnia angajeaza doi angajati cu aceeasi combinatie de nume si prenume. Desi ar fi posibila asocierea acestor campuri cu campuri aditionale, cum ar fi data angajarii, pentru a asigura unicitatea, aceasta metodologie ar incalca principiul minimalitatii in alegerea cheii primare. In plus, si principiul stabilitatii este intr-o oarecare masura incalcat, intrucat o angajata poate sa se casatoreasca si sa isi schimbe numele.
2. **CNP** - aceasta optiune ar putea fi valida, insa daca exista angajati straini este posibil ca acestia sa nu aiba cod numeric personal.

Exemplele de mai sus cauta sa identifice o cheie primara naturala (regasita printre campurile ce caracterizeaza un angajat). Insa aceste campuri se dovedesc nesatisfacatoare.

Solutia: introducerea unei **chei primare artificiale**; aceasta nu apare in mod „natural” in tabel, ci este construita de catre proiectantul bazei de date. In acest scop, se creeaza un camp nou care respecta toate elementele unei chei si se adauga in tabel.

ID	**Prenume client**	**Nume client**	**CNP**	**Telefon**	**Data Angajarii**
100	Ana	Ionescu	2820216627918	0711123456	02/15/2024
101	Mihai	Popescu	1920415647718	0789765345	08/01/2025
102	Bogdan	Florescu	1750212697717	0723674660	11/03/2026

Se recomanda introducerea unui astfel de camp identificator (camp ID) in toate tabelele create. Este un camp de dimensiuni mici si foarte stabil, de tip intreg si este foarte usor de procesat cand se lucreaza cu tabelul respectiv.

Cheia externa (foreign key - FK)

Doua tabele sunt conectate atunci cand exista o dependenta logica intre ele, iar aceasta relatie se stabileste preluand o copie primara a cheii primare din primul tabel si incorporand-o in structura celui de-al doilea tabel, unde se transforma in cheie externa. Numele de cheie externa deriva din faptul ca al doilea tabel are deja o cheie primara proprie, iar cheia primara introdusa din primul tabel este 'externa' pentru al doilea tabel.

Astfel este mentinuta integritatea referențială, adica nu poate fi introdusa o comandă pentru un client inexistent.

IMPRESARI

Identificator impresar	Prenume impresar	Nume impresar	Data angajarii	Telefon mobil
100	Radu	Groza	02/16/2001	0744.444.111
101	Anca	Pop	09/02/2006	0722.000.111
102	Florin	Ionescu	05/23/2003	0723.010.110

Cheie primara

Cheie externa

ARTISTI

Identificator artist	Identificator impresar	Nume artist	....alte campuri.....
8001	100	Blondi	
8002	101	Morandi	
8003	100	Iris	

Identificator impresar este cheia primara a tabelului IMPRESARI si o cheie externa in tabelul ARTISTI. Campul *Identificator impresar* isi asuma acest rol deoarece tabelul ARTISTI are deja o cheie primara, si anume *Identificator artist*. Deci campul *Identificator impresar* stabileste relatia intre cele doua tabele.

De retinut!

- Tipul de date si dimensiunea cheii externe trebuie sa fie identice cu ale cheii primare.
- Nu este obligatoriu ca numele sa fie identic (ID_Produs in tabelul Produse poate fi denumit Cod_Produs in tabelul Comenzi)
- Stergerea unei inregistrari din tabelul parinte, trebuie sa conectata la comportamentul pentru inregistrarile copil (va fi definit ulterior in laborator).

II.3 Normalizarea tabelelor. Forme normale

Normalizarea datelor poate fi privita ca un proces de organizaree a datelor, in timpul caruia tabelele nesatisfacatoare sunt descompuse in mai multe tabele prin separarea atributelor astfel incat sa satisfaca proprietatile dorite.

In aceasta sectiune se trateaza procesul normalizarii si primele trei forme normale pentru un tabel.

Procesul de normalizare propus de E.F. Codd in 1970 urmareste executia unor serii de teste asupra unui tabel, pentru a stabili apartenenta sa la o anumita forma normala. Codd propune trei forme normale (3NF), cea mai buna definitie fiind data mai tarziu de Boyce si Codd, fiind cunoscuta sub numele de forma normala Boyce-Codd.

In fond, unul din obiectivele procesului de normalizare este asigurarea faptului ca baza de date este bine construita asigurandu-se consistenta la modificare, adaugare si stergere, cat si eliminarea anomaliilor la operatiile de interogare.

Forma normala de ordin 1 (FN1)

Forma normala de ordin 1 este considerata ca fiind parte a definitiei formale a unui tabel.

Ea nu permite atribute cu mai multe valori, atribute compuse sau combinatii ale lor. Aceasta stabileste ca domeniul atributelor trebuie sa includa numai valori atomice si valoarea oricarui atribut intr-un n-uplu este o valoare unica in domeniul atributului respectiv. Deci, FN1 nu permite un set de valori, un n-uplu de valori sau o combinatie a acestora ca valoare a unui atribut pentru un n-uplu.

Cu alte cuvinte, FN1 nu permite tabele in tabele sau tabele ca atribute ale n-uplurilor. Valorile permise de FN1 sunt atomice sau indivizibile, pentru un domeniu specificat de valori.

Exemplu:

Se considera ca in tabelul Materii (*MaterieID, Denumire, An, NumeProfesor*), unde cheia primara este *MaterieID* este introdusa o inregistrare de tipul:

MaterieID	**Denumire**	**An**	**NumeProfesor**
1	Analiză matematică	1	O.Stanasila, P.Flondor, M.Olteanu

Aceasta inregistrare ar dori sa indice ca o disciplina care este predata de trei profesori diferiti. Acesta inregistrare **nu indeplineste FN1**, deoarece pentru atributul *NumeProfesor* nu sunt definite valori atomice, ci un set de valori.

Pentru a rezolva aceasta problema vom introduce mai multe inregistrari, care vor indeplini cerintele FN1, considerand trei materii diferite, identificate prin *MaterieID* astfel:

MaterieID	**Denumire**	**An**	**NumeProfesor**
1	Analiză matematică	1	O.Stanasila
2	Analiză matematică	1	P.Flondor
3	Analiză matematică	1	M.Olteanu

Pentru a aduce tabelul la forma complet corectă, trebuie sa fie separat și numele profesorului, rezultând:

MaterieID	**Denumire**	**An**	**NumeProfesor**	**InițialăProfesor**
1	Analiză matematică	1	Stanasila	O.
2	Analiză matematică	1	Flondor	P.
3	Analiză matematică	1	Olteanu	M.

Forma normala de ordin 2 (FN2)

A doua forma normala impune ca fiecare atribut (coloana) sa fie dependent de fiecare parte a cheii principale.

Mai exact, un tabel indeplineste FN2 daca indeplineste FN1 si creeaza tabele separate pentru seturi de valori care apar in mai multe inregistrari. Apoi, creeaza relatii intre aceste noi tabele si celelalte tabele prin chei primare – chei externe.

Exemplu:

Intr-un tabel in care exista o cheie primara compusa (formata din doua sau mai multe campuri), fiecare atribut care nu este cheie trebuie sa depinda de intreaga cheie primara, nu doar de unul dintre campuri. In cazul in care depinde doar de unul dintre campuri, acel tabel trebuie separat in mai multe tabele, pentru a indeplini FN2.

Forma normala de ordin 3 (FN3)

Pentru a ajunge la a treia forma normala, tabelul trebuie sa indeplineasca conditiile de forma normala de ordinul 1 si 2. Pentru a indeplini conditia de forma normala de ordin 3, trebuie ca toate campurile non-primare sa depinda numai de campurile primare.

Desi nu face parte in mod riguros din normalizare, de obicei nu este recomandata includerea campurilor care pot fi derivate din alte campuri situate in acelasi tabel sau in tabelele cu care este in relatie.

Forma normala Boyce-Codd (FNBC)

Forma normala Boyce-Codd este o forma stricta FN3, intelegand prin aceasta ca fiecare tabela FNBC este in acelasi timp o tabela FN3, cu toate ca o tabela FN3 nu este in mod necesar si o tabela FNBC. Cele doua forme sunt asemanatoare, ambele impunand conditia ca atributul care determina functional alte atribute sa fie o cheie a tabelei. Forma normala Boyce-Codd este mai restrictiva decat FN3, deoarece in FNBC se impune aceasta conditie tuturor atributelor, prime sau neprime, pe cand in FN3 conditia se impune numai atributelor neprime. Atributele prime sunt atributele care apartin unei chei, iar celelalte se numesc atribute neprime.

Orice tabel format din doua coloane este FNBC, FN2 si FN3.

Acest tabel compus din doua atribute este FN2, deoarece fie cheia este formata din ambele atribute si atunci nu exista atribute neprime, fie cheia este formata dintr-unul din atribute, iar dependenta functionala a celuilalt atribut (care este atribut neprim) fata de cheie este totala.

Acest tabel compus din doua atribute este FN3 deoarece este FN2 si nu poate exista nici un atribut neprim care sa determine functional un alt atribut neprim, deoarece un tabel cu doua atribute nu poate avea decat cel mult un atribut neprim.

II.3 Relatiile intre tabele

Intre doua tabele exista o relatie atunci cand este posibila stabilirea unei asocieri intre inregistrarile din primul tabel si inregistrarile celui de-al doilea tabel. Relatia stabileste o conexiune intre doua tabele care sunt corelate din punct de vedere logic si reprezinta mecanismul care permite extragerea datelor din mai multe tabele simultan.

Exemplu:

In baza de date folosita pentru gestiunea unei scoli de muzica apar urmatoarele doua tabele, Studenti si Instrumente studenti:

STUDENTI

Student ID	Prenume	Nume	<<alte campuri>>
201	Mihai	Ilea	.....
202	Lavinia	Mihai	.....
203	Alexandru	Tomescu	.....
204	Andrei	Popa	.....
205	Ioana	Aldea	.....
			.....

INSTRUMENTE STUDENTI

Student ID	Instrument ID	Data imprumut
202	100	26/04/08
201	102	12/05/08
203	110	15/05/08
203	113	18/06/08
203	111	03/07/08
101	122	02/09/08
101	151	15/10/08

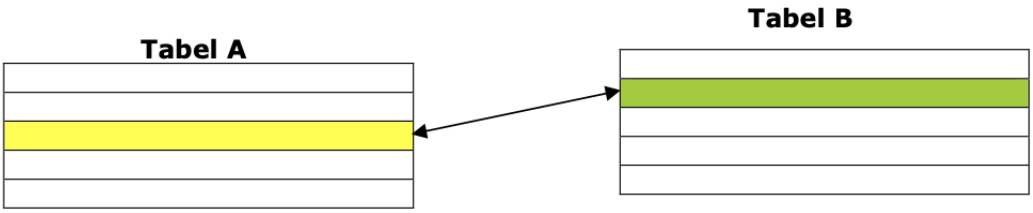
Exista o relatie logica intre datele celor doua tabele. Pe parcursul unui an scolar, un student poate imprumuta unul sau mai multe instrumente din dotarea scolii. Asta se traduce prin faptul ca o inregistrare din tabelul Studenti (reprezentand studentul) poate fi corelata cu una sau mai multe inregistrari din tabelul Instrumente studenti (fiecare reprezentand un instrument pe care il imprumuta studentul).

Tipuri de relatii

Exista trei tipuri de relatii care pot aparea intre doua tabele: unu cu unu (one-to-one), unu cu mai multi (one-to-many) si mai multi cu mai multi (many-to-many). Identificarea corecta a relatiilor care exista intre tabelele este esentiala pentru a proiecta cu succes o baza de date. Relatiile sunt cele care asigura eliminarea datelor duplicate si care asigura o redundanta minima a datelor.

Relatii de tip unu cu unu (one-to-one)

Doua tabele au o relatie de tip unu cu unu (one-to-one) cand o singura inregistrare din primul tabel este corelata cu o singura inregistrare din al doilea tabel si o singura inregistrare din al doilea tabel este corelata cu o singura inregistrare din primul tabel.



O situatie tipica in care regasim acest tip de relatie este intr-o baza de date folosita pentru gestiunea departamentului Personal dintr-o companie.

ANGAJATI

Angajat ID	Prenume	Nume	Telefon	<<alte campuri>>
100	Andrei	Ionescu	0722.01.01.01	.....
101	Andreea	Toma	0745.45.45.45	.....
102	Mihai	Popa	0766.00.11.22	.....

SALARII

Angajat ID	Salariu	<<alte campuri>>
100	545	.....
101	1030	.....
102	775	.....

Desi aceste date ar putea fi combinate intr-un singur tabel, explicatia pentru care se putea opta pentru structura prezentata mai sus ar fi urmatoarea:

Datele din tabelul Angajati trebuie sa fie accesibile oricarui angajat al companiei, in timp ce la datele din tabelul Salarii trebuie sa aiba acces numai anumite persoane autorizate.

Stabilirea in baza de date a unei relatii one-to-one

In acest tip de relatie, un tabel va juca rolul de tabel **parinte**, iar cel de-al doilea tabel va juca rolul de tabel **copil**.

In tabelul parinte trebuie sa existe o inregistrare inainte de a introduce in tabelul copil o **inregistrare corelata**. Modul in care sunt atribuite rolurile de parinte si copil tabelelor depinde de subiectele reprezentate de tabele.

In situatia prezentata mai sus, este nevoie ca mai intai sa existe o inregistrare a angajatului in tabelul ANGAJATI, si abia apoi sa ii fie definita o inregistrare corelata in tabelul SALARII (trebuie sa fie cunoscut identificatorul unic al angajatului, pentru a se alege cui i se atribuie salariul respectiv).

O relatie de tip unu cu unu se stabileste prin **preluarea unei copii a cheii primare** a tabelului parinte si **includerea** acesteia in structura **tabelului copil**, unde devine o **cheie externa**. In multe situatii, cheia externa va juca si rol de cheie primara a tabelului copil (vezi figura de mai sus).

Exemplu:

Avand instructura organizationala a unei companii, compania este impartita in mai multe departamente si fiecare departament este condus de un manager. Un angajat poate fi manager la un singur departament.

Pentru a reprezenta aceasta structura in baza de date vom folosi doua tabele, Departamente si Manageri, avand urmatoarele campuri:

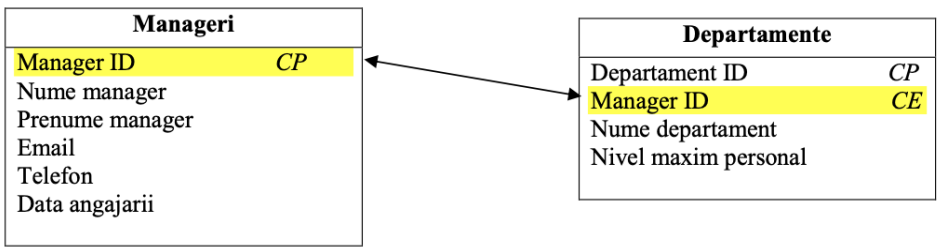
Departamente	
Departament ID	*CP*
Nume departament	
Nivel maxim personal	

Manageri	
Manager ID	*CP*
Nume manager	
Prenume manager	
Email	
Telefon	
Data angajarii	

Un departament poate fi condus de un singur manager la un moment dat, deci o inregistrare din tabelul Departamente poate fi corelata cu o singura inregistrare din tabelul Manageri.

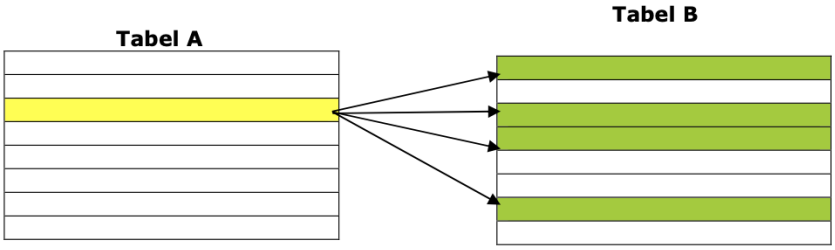
In sens invers, un manager poate conduce un singur departament la un moment dat, deci o inregistrare din tabelul Manageri poate fi corelata cu o singura inregistrare din tabelul Departamente. In concluzie, intre cele doua tabele exista o relatie one-to-one.

Conform retetei enuntate mai devreme, trebuie identificate tabelul parinte si tabelul copil. In acest caz, alegerea poate fi facuta arbitrar. De exemplu, se presupune ca Manageri este tabelul parinte si Departamente este tabelul copil. Pentru a stabili legatura, va fi preluata cheia primara din tabelul parinte, Manageri, si va fi introdusa in structura tabelului copil, Departamente.

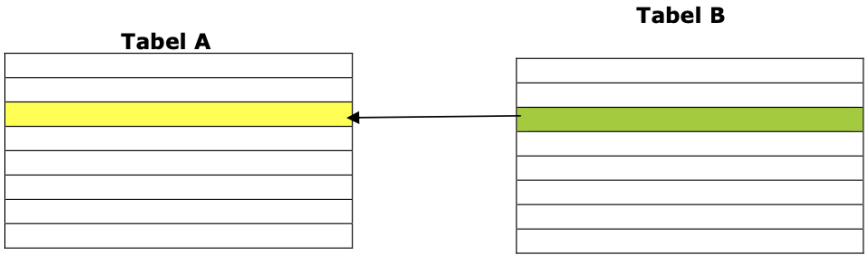


Relatii de tip unu cu mai multi (one-to-many)

Intre doua tabele exista o relatie de tip unu cu mai multi (one-to-many) atunci cand o inregistrare din primul tabel este corelata cu una sau mai multe inregistrari din al doilea tabel, insa o inregistrare din al doilea tabel este corelata cu o singura inregistrare din primul tabel.



Din perspectiva tabelului A, o inregistrare poate fi corelata cu una sau mai multe inregistrari din tabelul B, astfel tabelul B poate fi interpretat ca fiind partea ”mai multi” a relatiei.



In sens invers, o inregistrare din tabelul B poate fi corelata cu o singura in registrare din tabelul A. Ne putem referi la tabelul A ca fiind partea ”unu” relatiei.

Exemplu:

Acest tip de relatie este cel mai des intanit in proiectarea unei baze de date, fiind si cel mai usor de identificat. Un exemplu tipic de situatie avand relatii de tip unu cu mai multi intr-o baza de date este urmatorul:

STUDENTI

Student ID	Prenume	Nume	<<alte campuri>>
201	Mihai	Ilea	.....
202	Lavinia	Mihai	.....
203	Alexandru	Tomescu	.....
204	Andrei	Popa	.....
205	Ioana	Aldea	.....
			.....

INSTRUMENTE STUDENTI

Student ID	Instrument ID	Data imprumut
202	100	26/04/08
201	102	12/05/08
203	110	15/05/08
203	113	18/06/08
203	111	03/07/08
101	122	02/09/08
101	151	15/10/08

In baza de date a unei scoli de muzica, fiecare student poate fi identificat prin intermediul *Student ID*, iar fiecare instrument poate fi identificat cu *Instrument ID*. Un student poate imprumuta unul sau mai multe instrumente, astfel, tabelul INSTRUMENTE STUDENTI va deveni partea “mai multi” a relatiei. In sens invers, un instrument poate fi imprumutat la un moment dat de un singur student, astfel tabelul STUDENTI reprezinta partea “unu” a relatiei.

Stabilirea in baza de date a unei relatii one-to-many

In cazul acestui tip de relatie, primul pas consta in identificarea partii ”unu” si a partii ”mai multi”. Apoi, pentru a stabili legatura in structura bazei de date, trebuie sa fie preluata o copie a cheii primare din tabelul aflat in partea ”unu” si sa o introdusa in structura tabelului din partea ”mai multi”, unde aceasta va deveni cheie externa.

Exemplu:

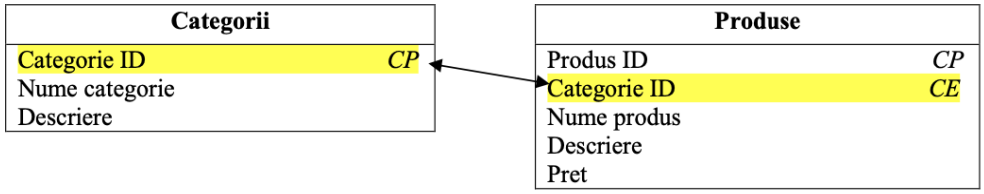
In baza de date a unui magazin exista produse care apartin mai multor categorii. Din specificatiile primite de la magazinul respectiv, un produs poate apartine unei singure categorii, iar o categorie contine unul sau mai mute produse. Pentru a reprezenta aceasta structura, vor fi folosite doua tabele, Categorii si Produse, avand urmatoarele campuri:

Categorii	
Categorie ID	*CP*
Nume categorie	
Descriere	

Produse	
Produs ID	*CP*
Nume produs	
Descriere	
Pret	

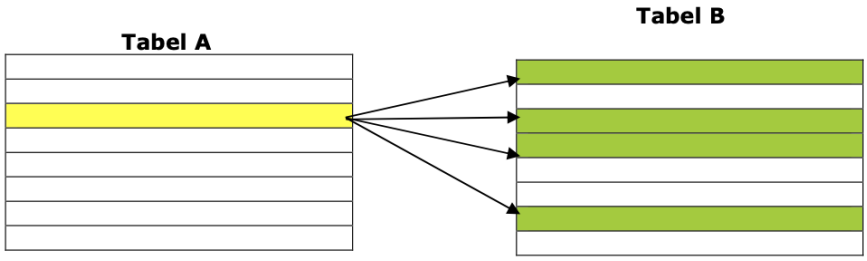
Un produs poate apartine unei singure categorii, deci o inregistrare din tabelul Produse poate fi corelata cu o singura inregistrare din tabelul Categorii. In sens invers, o categorie cuprinde unul sau mai multe produse, deci o inregistrare din tabelul Categorii poate fi corelata cu una sau mai multe inregistrari din tabelul Produse. In concluzie, intre cele doua tabele exista o relatie one-to-many.

Conform retetei enuntate mai devreme, are loc identificarea partii ”unu” a relatiei si a partii ”mai multi” a relatiei. Tabelul Categorii reprezinta partea ”unu”, iar tabelul Produse reprezinta partea ”mai multi”. Deci pentru a stabili legatura, va fi preluata cheia primara din tabelul Categorii (partea ”unu”), si o va fi introdusa in structura tabelului Produse (partea ”mai multi”).

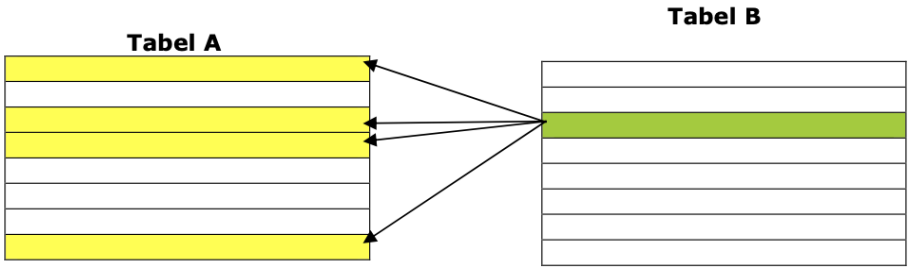


Relatii mai multi cu mai multi (many-to-many)

Intre doua tabele exista o relatie de tip mai multi cu mai multi (many-to- many) cand o inregistrare din primul tabel este corelata cu una sau mai multe inregistrari din al doilea tabel, si o inregistrare din al doilea tabel poate fi corelata cu una sau mai multe inregistrari din primul tabel.



Din perspectiva tabelului A, o inregistrare poate fi corelata cu una sau mai multe inregistrari din tabelul B.



In sens invers, o inregistrare din tabelul B poate fi corelata cu una sau mai multe inregistrari din tabelul A.

Acest tip de relatie este al doilea ca frecventa de aparitie in proiectarea unei baze de date si este ceva mai dificil de identificat decat o relatie de tip unu cu mai multi.

Exemplu:

In baza de date a unei facultati, pe parcursul unui an universitar, un student poate participa la unul sau mai multe cursuri, deci o inregistrare din tabelul Studenti poate fi corelata cu una sau mai multe inregistrari din tabelul Cursuri. Totodata, un curs este frecventat la un moment dat de unul sau mai multi studenti, deci o inregistrare din tabelul Cursuri este corelata cu una sau mai multe inregistrari din tabelul Studenti. Deci, intre aceste doua tabele exista o relatie de tip many-to-many.

STUDENTI

Student ID	Prenume	Nume	Oras	Cod postal	<<alte campuri>>
601	Adrian	Anghel	Bucuresti	262788	.....
602	Mircea	Badea	Cluj	213312	.....
603	Nicoleta	Avram	Brasov	211009	.....
604	Dorin	Ionescu	Bucuresti	252622	.....
605	Maria	Radu	Craiova	290980	.....

CURSURI

Curs ID	Nume curs	Credite	Profesor ID	Sala curs	<<alte campuri>>
51	Introducere in informatica	4	25	EC101	.....
52	Dispozitive si circuite electronice	6	31	EC105	.....
53	Analiza matematica	5	78	EC001	.....
54	Baze de date	5	43	EC101	.....
55	Inteligenta artificiala	4	12	EC004	.....
56	Programare orientata pe obiecte	4	55	A250	.....

Stabilirea in baza de date a unei relatii many-to-many

O relatie de tip mai multi cu mai multi se stabileste cu ajutorul unui **tabel de legatura**. Tabelul de legatura se defineste luand copii ale cheilor primare din fiecare tabel si utilizand aceste chei pentru a forma structura tabelului. In tabelul de legatura, aceste campuri indeplinesc doua roluri:

- impreuna, ele constituie cheia primara compusa a tabelului
- fiecare este o cheie externa unica, care ajuta la stabilirea unei relatii intre tabelul sau parinte si tabelul de legatura

Exemplu:

Folosind situatia prezentata mai sus, intr-o universitate exista studenti (tabelul Studenti) care urmeaza cursuri (tabelul Cursuri).

Studenti	
Student ID	*CP*
Prenume student	
Nume student	
Oras	
Cod postal	
<<alte campuri>>	

Cursuri	
Curs ID	*CP*
Nume curs	
Credite	
Profesor ID	
Sala curs	
<<alte campuri>>	

Conform retetei enuntate mai devreme, va fi folosit un tabel de legatura. Pentru a forma structura de baza a acestui tabel, trebuie preluate cheile primare din cele doua tabele intre care se stabileste relatia: Student ID si Curs ID. Impreuna, aceste doua campuri vor reprezenta cheia primara a tabelului de legatura. Ca in cazul oricarui tabel, si un tabel de legatura poate contine si o serie de alte campuri, reprezentative pentru fiecare asociere dintre tabelele pe care le uneste. De exemplu, fiecare asociere intre un student si un curs poate avea o data specifica de inceput si de incheiere.

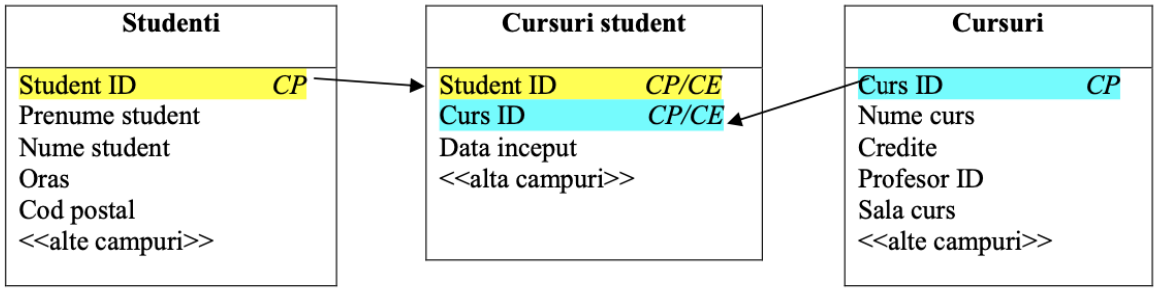
Cursuri student	
Student ID	*CP/CE*
Curs ID	*CP/CE*
Data inceput	
<<alte campuri>>	

Explicatii:

Relatia initiala “mai multi cu mai multi” a fost dizolvata, deoarece nu mai exista o relatie directa intre tabelele **Studenti** si **Cursuri**, fiind inlocuita cu doua relatii unu cu mai multi: una intre tabelele **Studenti** si **Cursuri Student** si alta intre tabelele **Cursuri** si **Cursuri Student**.

In prima relatie, o inregistrare din tabelul **Studenti** poate fi asociata cu una sau mai multe inregistrari din tabelul **Cursuri Student**, insa o inregistrare din tabelul **Cursuri** STUDENT nu poate fi corelata decat cu o singura inregistrare din tabelul **Studenti**.

In a doua relatie, o inregistrare din tabelul **Cursuri** poate fi corelata cu una sau mai multe inregistrari din tabelul **Cursuri Student**, in timp ce o inregistrare din tabelul **Cursuri Student** poate fi corelata cu o singura inregistrare din tabelul **Cursuri**.



- Tabelul de legatura CURSURI STUDENT contine doua chei externe: campurile Student ID (tabelul STUDENTI) si Curs ID (tabelul CURSURI) sunt amandoua copii ale cheilor primare. Prin urmare, fiecare dintre ele este, prin definitie, cheie externa. Ele ajuta la stabilirea relatiei intre tabelele lor parinte si tabelul de legatura.
- Tabelul de legatura are o **cheie primara compusa**, formata din campurile **Student ID** si **Curs ID**.
- Tabelul de legatura ajuta la pastrarea datelor redundante la un minim absolut. O astfel de structura permite asocierea unui student cu oricat de putine sau de multe cursuri.

Relatii cu autoreferire

Acest tip particular de relatie nu se stabileste intre doua tabele, ci intre inregistrarile dintr-un tabel. Un tabel are cu el insusi o relatie cu autoreferire (cunoscuta si sub numele de relatie recursiva) daca o inregistrare din tabel este corelata cu alte inregistrari din acelasi tabel.

Ca si in cazul relatiilor care se stabilesc intre doua tabele, o relatie cu autoreferire poate fi:

- Unu cu unu: cand o inregistrare dintr-un tabel poate fi corelata cu o singura alta inregistrare din acelasi tabel.
- Unu cu mai multi: cand o inregistrare dintr-un tabel poate fi corelata cu una sau mai mult inregistrari din acelasi tabel.
- Mai multi cu mai multi: cand o inregistrare dintr-un tabel poate fi corelata cu una sau mai mult inregistrari din acelasi tabel si una sau mai multe inregistrari pot fi corelate cu inregistrarea data.

Exemplu:

In contextul unei organizatii, o relatie de autoreferire poate aparea atunci cand angajatii formeaza o ierarhie, mai exact un angajat poate fi si manager pentru alti angajati, dar poate avea si un manager superior. Astfel, tabelul ANGAJATI se leaga singur printr-o cheie externa, pentru a permite relatia intre manageri si echipele de angajati din subordine.

ID_Angajat	**Nume**	**ID_Manager**
1	Ionescu I.	NULL
2	Popescu A.	1
3	Dumitrescu O.	1
4	Popescu M.	2

Explicatii:

- angajatul Ionescu I. nu are manager (ID_Manager = NULL) => el poate fi seful organizatiei/echipei
- angajatii Popescu A. si Dumitescu O. il au ca manager pe angajatul cu ID_Manager = 1, adica pe Ionescu I.
- angajatul Popescu M. il are ca manager pe angajatul cu ID_Manager = 2, adica ii raporteaza lui Popescu A.

Astfel de constrangeri permit mentinerea integritatii referentiale, adica nu se poate atribui un manager inexistent.

Integritatea datelor

Prin integritatea datelor se intelege validitatea, consecventa si acuratetea datelor incluse intr-o baza de date.

Exista 4 tipuri de integritate a datelor, implementabile pe parcursul procesului de proiectare. Trei dintre acestea se bazeaza pe diferite aspecte ale structurii bazei de date si sunt denumite in conformitate cu nivelul la care opereaza. Cel de-al patrulea tip de integritate a datelor se bazeaza pe modul in care o organizatie isi percepe si isi utilizeaza datele.

- integritatea la **nivel de tabel** (integritatea entitatii) - asigura lipsa inregistrarilor duplicate in interiorul tabelului si faptul ca acel camp care identifica fiecare inregistrare din tabel (cheia primara) este unic si nu are valori nule.
- integritatea la **nivel de camp** (integritatea domeniului) - asigura faptul ca structura fiecarui camp este solida, ca valorile din fiecare camp sunt valide, consecvente si precise, precum si ca se asigura o definire consecventa a campurilor de acelasi tip (Oras, de exemplu).
- integritatea la **nivel de referinta** (integritatea referentiala) - asigura soliditatea relatiei dintre 2 tabele, precum si faptul ca inregistrarile din tabele sunt sincronizate ori de cate ori datele sunt introduse, actualizate sau sterse din oricare dintre cele 2 tabele.
- **regulile de desfasurare** a activitatii - impun restrictii sau limitari asupra anumitor aspecte ale unei baze de date, pe baza modalitatilor in care o organizatie isi percepe si utilizeaza datele.

III Studiu de caz: Companie

Pentru a exemplifica si a aplica direct notiunile din acest indrumar, va fi folosit ca studiu de caz o baza de date, avand urmatoarea poveste:

Prietenul meu, Andrei, a fost de curand numit intr-un post de conducere la o companie de nivel mediu, al carei ritm de crestere este constant. Odata numit in noua pozitie, Andrei a constatat ca gestiunea companiei este tinuta folosind foi de calcul tabelar, sau, mai rau, pe hartie. In consecinta, companie pierde o gramada de timp cu intretinerea acestor date. Andrei stie ca utilizarea unei baze de date reprezinta o buna modalitate de stocare si manipulare a datelor asociate companiei sale. Utilizarea unei baze de date va reduce semnificativ timpul pe care il va aloca intretinerii datelor, iar el poate sa se asigure in orice moment ca datele au fost actualizate si ca au fost corect introduse. In acest sens, Andrei a decis sa contacteze un consultant de specialitate care sa-i proiecteze si implementeze baza de date.

In aceasta poveste, tu esti specialistul cu care vom colabora pentru proiect. In primul rand, trebuie sa stam de vorba cu Andrei si sa aflam cum functioneaza compania in cauza.

Din discutia cu el, aflam urmatoarele:

- Compania se ocupa de dezvoltare de software si are ca scop realizarea unor proiecte.
- Compania este organizata in departamente. Fiecare departament este caracterizat de un nume si un cod de identificare. Fiecare departament este condus de un manager.
- Compania are un numar de angajati organizati pe departamente. Fiecare angajat poate apartine unui singur departament. La un moment dat, un angajat poate fi managerul unui singur departament. De asemenea, fiecare angajat are un sef direct, numit si supervizor.
- Pentru fiecare angajat se retin urmatoarele date: nume, prenume, CNP, adresa, sex, data nasterii si salariul. De asemenea, fiecare angajat este afiliat unui departament.
- Compania are ca scop realizarea unor proiecte. Fiecare proiect este caracterizat de un nume, un numar de cod, un departament care il coordoneaza, un buget alocat si un termen limita.
- Un departament poate fi implicat in mai multe proiecte la un moment dat.
- Orice angajat este afiliat la un departament, insa poate lucra la mai multe proiecte, ce nu sunt neaparat coordonate de acelasi departament. Se stocheaza numarul de ore alocate saptamanal pentru fiecare proiect.
- Lista persoanelor aflate in intretinerea fiecarui angajat este importanta, intrucat este utilizata la calculul impozitului. Lista contine numele, sex, data nasterii. Un intretinut poate fi asociat cu un singur angajat.

Pe baza acestor informatii, trebuie sa proiectam baza de date pentru compania lui Andrei.

Rezolvare:

Pasul 1: Identificarea tabelelor

Pentru acest pas, este importanta identificarea corecta a entitatilor care compun sistemul in lumea reala. Apoi, fiecare entitate va fi reprezentata in baza de date prin intermediul unui tabel.

Din cele relatate de Andrei, pot fi identificate urmatoarele entitati cu care lucreaza compania:

- Angajati
- Departamente
- Proiecte
- Intretinuti

Deci, pentru fiecare dintre aceste entitati va fi creat un tabel in baza de date Companie; in total, patru tabele initiale.

Pasul 2: Identificarea campurilor

In lumea reala, fiecare entitate este caracterizata de o serie de atribute. Spre exemplu, o persoana este caracteriza de un nume, prenume, CNP, adresa, etc., unice pentru fiecare individ.

In descrirea sa, Andrei a oferit detalii despre datele pe care compania este interesata sa le retina despre fiecare entitate. Pornind de la aceste informatii, vor fi identificate atributele caracteristice pentru fiecare entitate. Apoi, fiecare dintre aceste atribute va deveni un camp al tabelului corespunzator.

Angajati
Nume
Prenume
CNP
Strada
Numar
Oras
Judet/Sector
Sex
Data nasterii
Salariu

Departamente
Nume departament
Cod departament

Proiecte
Nume proiect
Cod proiect
Buget
Termen limita

Intretinuti
Nume
Prenume
Sex
Data nasterii

Pasul 3: Stabilirea cheilor primare

Pentru identificarea in mod unic a fiecarei inregistrari, va fi folosita o cheie primara artificiala. Astfel, pentru fiecare tabel identificat, indiferent de datele „naturale” pe care le contine, va fi introdus un camp identificator de tip intreg, avand rolul de cheie primara (ID).

Campurile Angajat ID, Departament ID, Proiect ID si Intretinut ID vor deveni cheile primare ale tabelelor din baza de date Companie. De regula, atribuirea valorilor cheii primare se lasa in seama sistemului de gestiune a bazelor de date (SGBD). Tabelele actualizate vor arata astfel:

Angajati
Angajat ID *CP*
Nume
Prenume
CNP
Strada
Numar
Oras
Judet/Sector
Sex
Data nasterii
Salariu

Departamente
Departament ID *CP*
Nume departament
Cod departament

Proiecte
Proiect ID *CP*
Nume proiect
Cod proiect
Buget
Termen limita

Intretinuti
Intretinut ID *CP*
Nume
Prenume
Sex
Data nasterii

Pasul 4: Identificarea relatiilor intre tabele

Pentru a identifica relatiile care se stabilesc intre tabele, poate fi folosita o matrice de tipul:

	Angajati	**Departamente**	**Proiecte**	**Intretinuti**
Angajati	1:1	1: 1	1:N	1:N
Departamente	1:N		1:N	
Proiecte	1:N	1:1		
Intretinuti	1:1			

Sunt folosite urmatoarele notatii, pentru a defini tipul de relatii intre entitati:

- 1:1 – pentru relatiile de tip unu cu unu
- 1:N – pentru relatiile de tip unu cu mai multi
- N:N – pentru relatiile de tip mai multi cu mai multi

Angajati

Din perspectiva tabelului ANGAJATI, sunt identificate urmatoarele tabele cu care relationeaza:

- Angajati – Angajati: fiecare angajat are un sef direct, numit si supervizor. Deci tabelul Angajati are o relatie cu autoreferire cu el insusi. Un angajat are un singur supervizor, deci relatia este de tip unu cu unu (1:1).
- Angajati – Departamente: Un angajat este afiliat unui singur departament, deci din perspectiva tabelului Angajati, exista o relatie de tip unu cu unu intre aceste doua tabele (1:1).

- Angajati – Proiecte: Un angajat poate lucra la mai multe proiecte in acelasi timp, deci din perspectiva tabelului Angajati, exista o relatie de tip unu cu mai multi intre aceste doua tabele (1:N).
- Angajati – Intretinuti: Un angajat poate avea unul sau mai multi intertinuti, deci din perspectiva tabelului Angajati, exista o relatie de tip unu cu mai multi intre aceste doua tabele (1:N).

Departamente

Din perspectiva tabelului DEPARTAMENTE, sunt identificate urmatoarele tabele cu care relationeaza:

- Departamente – Angajati: Un departament are mai multi angajati, deci din perspectiva tabelului Departamente, exista o relatie de tip unu cu mai multi intre aceste doua tabele (1:N).
- Departamente – Departamente: nu exista o relationare specificata
- Departamente – Proiecte: Un departament poate coordona unul sau mai multe proiecte la un moment dat, deci din perspectiva tabelului Departamente, exista o relatie de tip unu cu mai multi intre aceste doua tabele (1:N).
- Departamente – Intretinuti: nu exista o relationare specificata

Proiecte

Din perspectiva tabelului PROIECTE, sunt identificate urmatoarele tabele cu care relationeaza:

- Proiecte – Angajati: La un proiect lucreaza unul sau mai multi angajati, deci din perspectiva tabelului Proiecte, exista o relatie de tip unu cu mai multi intre aceste doua tabele (1:N).
- Proiecte – Departamente: Un proiect este coordonat de un singur departament, deci din perspectiva tabelului Proiecte, exista o relatie de tip unu cu unu intre aceste doua tabele (1:1).
- Proiecte – Proiecte: nu exista o relationare specificata
- Proiecte – Intretinuti: nu exista o relationare specificata

Intretinuti

Din perspectiva tabelului INTRETINUTI, sunt identificate urmatoarele tabele cu care relationeaza:

- Intretinuti – Angajati: Un intretinut este asociat cu un singur anjagat, deci din perspectiva tabelului Intretinuti, exista o relatie de tip unu cu unu intre aceste doua tabele (1:1).
- Intretinuti – Departamente: nu exista o relationare specificata
- Intretinuti – Proiecte: nu exista o relationare specificata
- Intretinuti – Intretinuti: nu exista o relationare specificata

Observatii

Deseori, relatiile difera de la o perspectiva la alta si este importanta etapa de determinare a tipului oficial de relatie care exista intre fiecare pereche de tabele din matrice. Acest lucru poate fi stabilit utilizand urmatorul set de formule:

1:1 + 1:1 = 1:1	Doua tabele au o relatie de tip unu cu unu (one- to-one) cand o singura inregistrare din primul tabel este corelata cu o singura inregistrare din al doilea tabel si o singura inregistrare din al doilea tabel este corelata cu o singura inregistrare din primul tabel.
1:N + 1:1 = 1:N 1:1 + 1:N = 1:N	Intre doua tabele exista o relatie de tip unu cu mai multi (one-to-many) cand o inregistrare din primul tabel este corelata cu una sau mai multe inregistrari din al doilea tabel, insa o inregistrare din al doilea tabel este corelata cu o singura inregistrare din primul tabel.
1:N + 1:N = N:N	Intre doua tabele exista o relatie de tip mai multi cu mai multi (many-to-many) cand o inregistrare din primul tabel este corelata cu una sau mai multe inregistrari din al doilea tabel, si o inregistrare din al doilea tabel poate fi corelata cu una sau mai multe inregistrari din primul tabel.

Utilizand aceste formule, relatiile oficiale care exista intre tabelele bazei de date Companie sunt:

- Angajati – Angajati: 1:1 + 1:1 => 1: 1 (relatie cu autoreferire)
- Angajati – Departamente: 1:1 + 1:N => 1:N
- Angajati – Proiecte: 1:N + 1:N => N:N
- Angajati – Intretinuti: 1:N + 1:1 => 1:N
- Proiecte – Departamente 1:N + 1:1 => 1:N

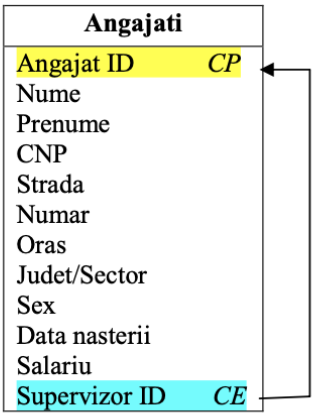
Pasul 5: Stabilirea relatiilor intre tabele

Odata identificate relatiile dintre tabele, acestea trebuie realizate si in baza de date.

Angajati – Angajati: 1:1 + 1:1 = 1: 1 (relatie cu autoreferire)

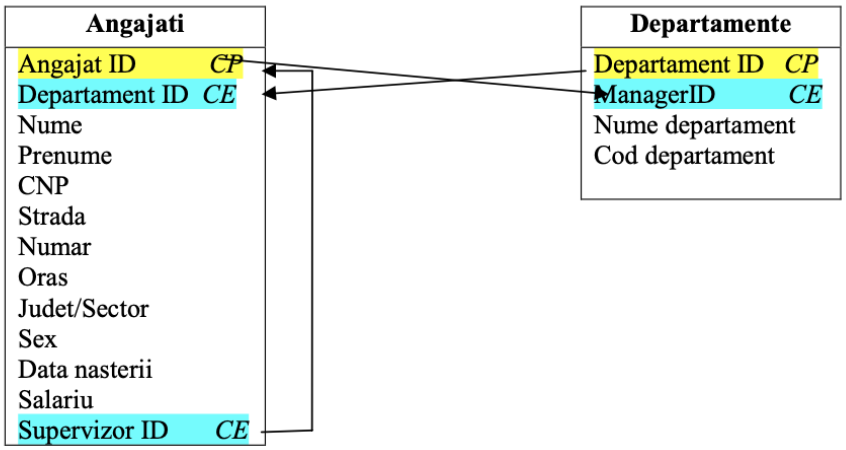
Conform retetei, o relatie one-to-one se stabileste prin preluarea unei copii a cheii primare a tabelului parinte si includerea acesteia in structura tabelului copil, unde devine o cheie externa.

Tinand cont ca aici este vorba despre o relatie cu autoreferire, tabelul parinte coincide cu tabelul copil. Deci, in structura tabelului Angajati introducem o copie a cheii primare Angajat ID, pe care o vom denumi pentru claritate Supervizor ID. Valoarea acestui camp va indica, pentru fiecare angajat in parte, angajatul in fata caruia raspunde.



Angajati – Departamente: 1:1 + 1:N = 1:N

Conform retetei, pentru a stabili o relatie one-to-many, primul pas este identificarea partii „unu” si a partii „mai multi”. In acest caz, intrucat un angajat poate fi afiliat unui singur departament, iar un departament poate avea unul sau mai multi angajati, partea „unu” este reprezentata de tabelul Departamente, iar partea „mai multi” este reprezentata de tabelul Angajati. Deci, pentru a reprezenta relatia in baza de date, va fi preluata o copie a cheii primare din tabelul aflat in partea ”unu”, adica Departamente, si va fi introdusa in structura tabelului din partea ”mai multi”, adica Angajati, unde aceasta va deveni cheie externa.

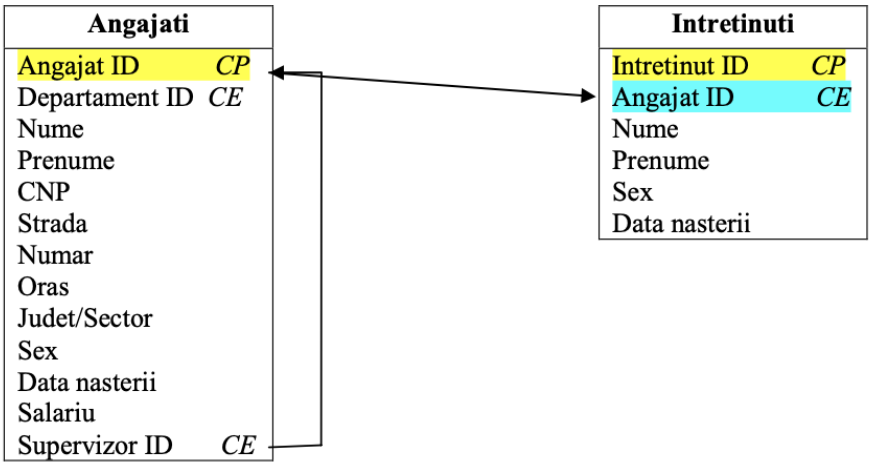


Totodata, intre aceste doua tabele mai exista o relatie de tip unu cu unu (1:1) data de faptul ca fiecare departament este condus de unul dintre angajatii companiei. Un angajat nu poate conduce decat un departament. Pentru a reprezenta aceasta relatie in baza de date, va fi preluata o copie a cheii primare din tabelul Angajati si introdusa in structura tabelului Departamente,

unde va deveni cheie externa. Pentru mai multa claritate, acest camp este denumit ManagerID in tabelul Departamente.

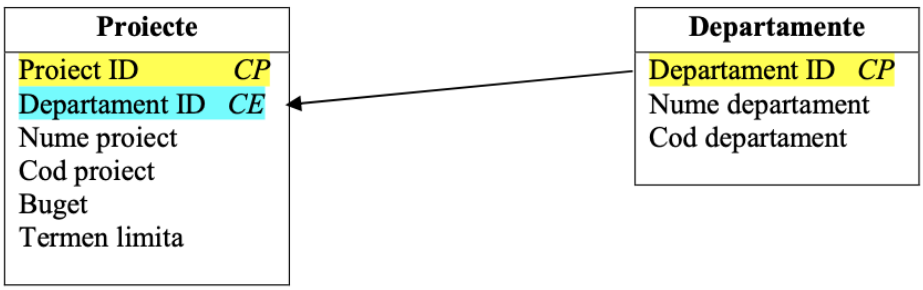
Angajati – Intretinuti: 1:N + 1:1 = 1:N

Conform retetei, pentru a stabili o relatie unu cu mai multi, primul pas identificarea partii „unu" si a partii „mai multi". In acest caz, intrucat un angajat poate avea mai multi intretinuti in ingrijire, iar un intretinut poate fi corelat cu un singur angajat, partea „unu" este reprezentata de tabelul Angajati, iar partea „mai multi" este reprezentata de tabelul Intretinuti. Deci, pentru a reprezenta relatia in baza de date, va fi preluata o copie a cheii primare din tabelul aflat in partea "unu", adica Angajati, si introdusa in structura tabelului din partea "mai multi", adica Intretinuti, unde aceasta va deveni cheie externa.



Proiecte – Departamente 1:N + 1:1 = 1:N

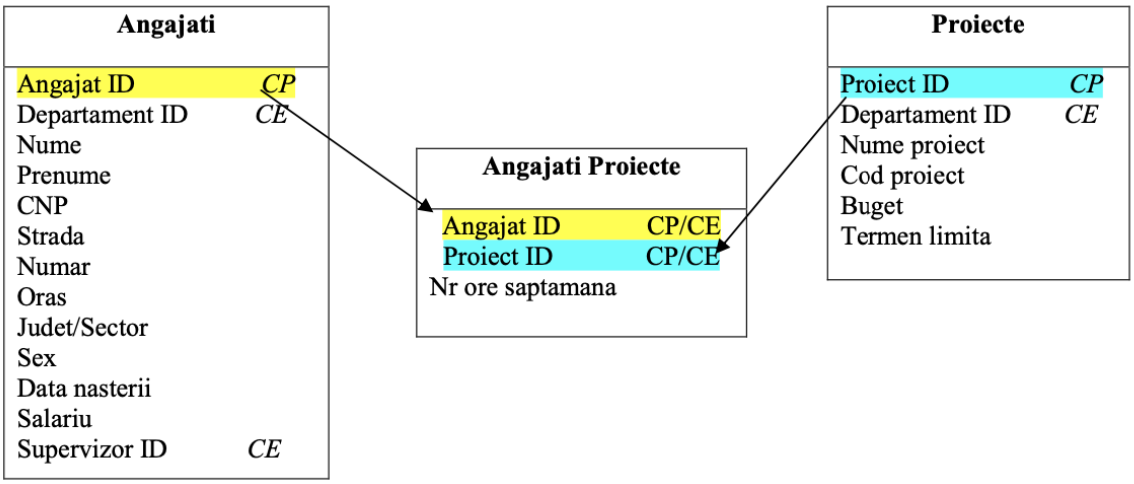
Conform retetei, pentru a stabili o relatie unu cu mai multi, primul pas este identificarea partii partea „unu" si a partii „mai multi". In acest caz, intrucat un proiect poate fi coordonat de un singur departament, iar un departament poate coordona unul sau mai multe proiecte, partea „unu" este reprezentata de tabelul Proiecte, iar partea „mai multi" este reprezentata de tabelul Departamente. Deci, pentru a reprezenta relatia in baza de date, va fi preluata o copie a cheii primare din tabelul aflat in partea "unu", adica Proiecte, si introdusa in structura tabelului din partea "mai multi", adica Departamente, unde aceasta va deveni cheie externa.



Angajati – Proiecte: 1:N + 1:N = N:N

Conform retetei, pentru a stabili o relatie mai multi cu mai multi, va fi folosi un tabel de legatura. Pentru a forma structura de baza a acestui tabel, trebuie extrase cheile primare din

cele doua tabele intre care a fost stabilita relatia: *Angajat ID* si *Proiect ID*. Impreuna, aceste doua campuri vor reprezenta cheia primara compusa a tabelului de legatura.

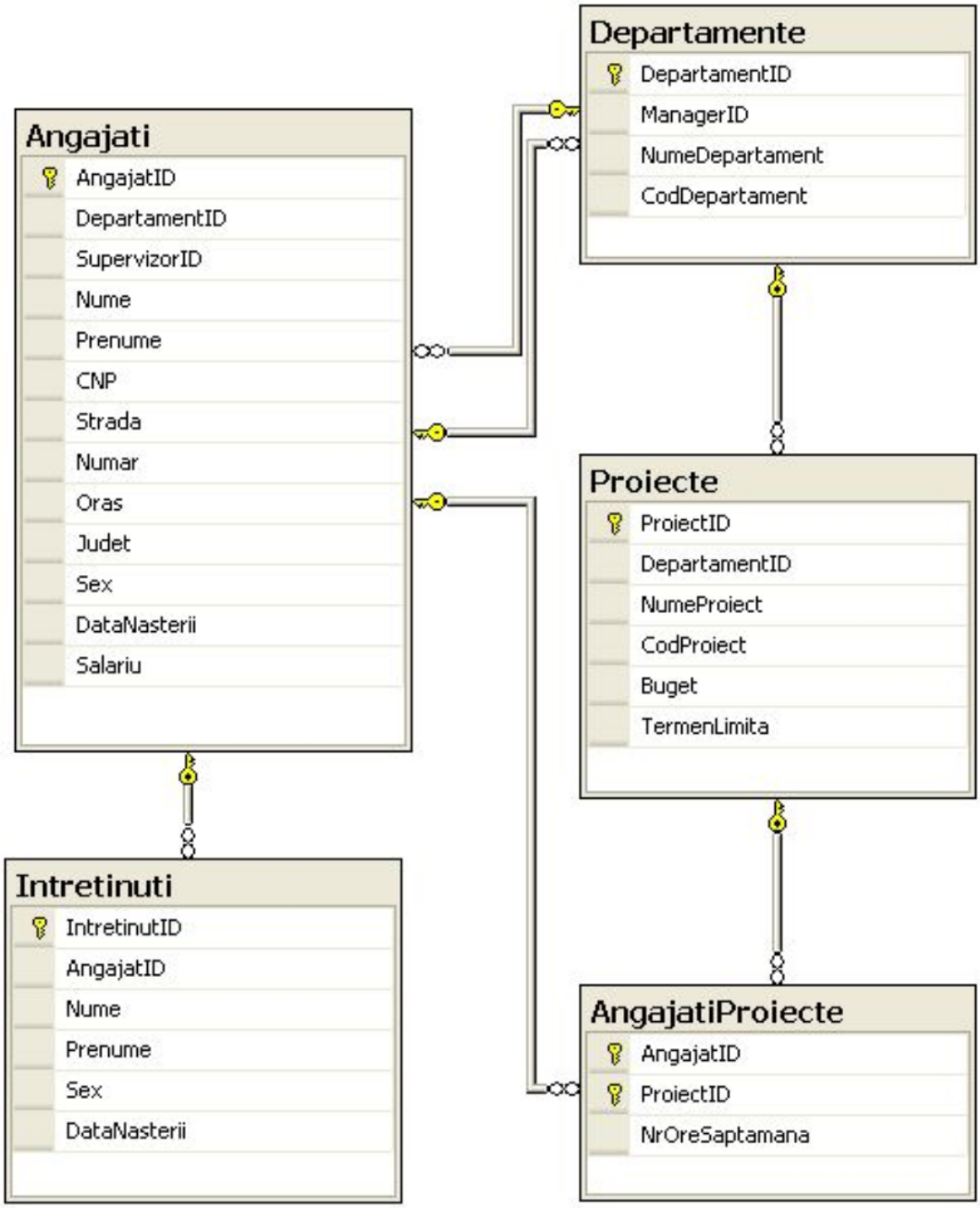


Tabelul de legatura Angajati Proiecte are o cheie primara compusa, formata din copii ale cheilor primare din cele doua tabele pe care le leaga: *Angajat ID* si *Proiect ID*.

Totodata, aceste doua campuri sunt si chei externe, care faciliteaza stabilirea unei relatii intre tabelele parinte (Angajati, respectiv Proiecte) si tabelul de legatura.

Dupa cum se poate observa si in acest caz, relatia mai multi cu mai multi a fost dizolvata in doua relatii unu cu mai multi: una intre tabelele Angajati si Angajati Proiecte, si cealalta intre tabelele Proiecte si Angajati Proiecte.

Diagrama bazei de date Companie arata astfel:


Departamente
DepartamentID
ManagerID
NumeDepartament
CodDepartament
Angajati
AngajatID
DepartamentID
SupervizorID
Nume
Prenume
CNP
Strada
Numar
Oras
Judet
Sex
DataNasterii
Salariu
Proiecte
ProiectID
DepartamentID
NumeProiect
CodProiect
Buget
TermenLimita
Intretinuti
IntretinutID
AngajatID
Nume
Prenume
Sex
DataNasterii
AngajatiProiecte
AngajatID
ProiectID
NrOreSaptamana

IV Intrebari de control

- Ce este o baza de date relationala?
- Care sunt caracteristicile unui tabel ideal?
- Care sunt caracteristicile unui camp ideal?
- Ce este o cheie primara?
- Ce este o cheie externa?
- Cum se stabileste cheia primara a unui tabel?
- Ce tipuri de relatii pot exista intre doua tabele?
- Care tip de relatie ridica cele mai multe probleme?
- Care sunt formulele folosite pentru identificarea tipului de relatie care exista intre doua tabele?
- Cum se stabileste in baza de date o relatie de tip unu cu unu?
- Cum se stabileste in baza de date o relatie de tip unu cu mai multi?
- Cum se stabileste in baza de date o relatie de tip mai multi cu mai
- multi?
- Ce este o relatie cu autoreferire?